

Selective Repeat Protocol

9 APRIL 2025

PREPARED BY

Ajay Singh (2023 BCS 004)
Abhinav Vats (2023 BCS 002)
Harshit Vishnoi (2023 BCS 025)
Prince Kumar (2023 BCS 050)
Shubham Sharma (2023 BCS 062)

The **selective repeat ARQ** is one of the Sliding Window Protocol strategies that is used where reliable in-order delivery of the data packets is required. The selective repeat ARQ is used for noisy channels or links and it manages the flow and error control between the sender and the receiver. In the selective repeat ARQ, we only resend the data frames that are damaged or lost. If any frame is lost or damaged then the receiver sends a negative acknowledgment (NACK) to the sender and if the frame is correctly received, it sends back an acknowledgment (ACK). As we only resend the selected damaged frames we name this technique the Selective Repeat ARQ technique. The ACK and the NACK have the sequence number of the frame that helps the sender identify the lost frame.

Key Features of Selective Repeat Protocol:

Sliding Window Mechanism:

SR utilizes a sliding window mechanism to manage the flow of packets between the sender and receiver. The sender maintains a window of packets that can be sent without waiting for acknowledgments (ACKs). The receiver also maintains a corresponding window to determine which packets it is ready to receive.

Selective Acknowledgment:

Unlike simpler ARQ protocols such as Go-Back-N, SR allows the receiver to acknowledge individual packets instead of cumulative ACKs. This selective acknowledgment reduces retransmission overhead by requiring only the retransmission of lost or corrupted packets.

Buffering at the Receiver:

The receiver buffers out-of-order packets until missing packets are retransmitted and received correctly. This buffering ensures that data is delivered to the application layer in the correct sequence.

Error Recovery:

When the receiver detects a missing or corrupted packet, it does not send a negative acknowledgment (NACK) explicitly. Instead, the absence of an acknowledgment for a specific packet prompts the sender to retransmit it after a timeout period.

Concurrency:

SR allows the sender to transmit multiple packets simultaneously, improving throughput in high-bandwidth, high-latency networks.

Windows: The Selective-Repeat protocol also uses two windows: a send window and a receive window. The send window maximum size can be $2^m - 1$. The receive window in Selective-Repeat is totally different from the one in Go-Back-N. The size of the receive window is the same as the size of the send window (maximum $2^m - 1$). The Selective-Repeat protocol allows as many packets as the size of the receive window to arrive out of order and be kept until there is a set of consecutive packets to be delivered to the application layer.

Sender: The sender starts in the ready state, but later it can be in one of the two states: ready or blocking.

Ready state. Four events may occur in this case:

If a request comes from the application layer, the sender creates a packet with the sequence number set to S_n . A copy of the packet is stored, and the packet is sent. If the timer is not running, the sender starts the timer. The value of S_n is now incremented, $S_n = (S_n + 1) \text{ modulo } 2^m$. If an error-free ACK, If there are outstanding packets, the timer is restarted; otherwise, the timer is stopped.

If a corrupted ACK or an error-free ACK with ackNo not related to an outstanding packet arrives, it is discarded. If a time-out occurs, the sender resends all unacknowledged packets in the window and restarts the timer.

Blocking state.

If an error-free ACK arrives with ackNo related to one of the outstanding packets, that packet is marked as acknowledged.

If a corrupted ACK or an error-free ACK with the ackNo not related to outstanding packets arrives, the ACK is discarded.

Receiver: The receiver is always in the ready state. Three events may occur: a. If an error-free packet with seqNo in the window arrives, the packet is stored and an ACK with ackNo = seqNo is sent. In addition, if the seqNo = Rn, then the packet and all previously arrived consecutive packets are delivered to the application layer and the window slides. b. If an error-free packet with seqNo outside the window arrives, the packet is discarded, but an ACK with ackNo = Rn is returned to the sender. c. If a corrupted packet arrives, the packet is discarded.

PROJECT IMPLEMENTATION

Overview:

The goal of this project was to simulate the Selective Repeat protocol and demonstrate its functioning through a custom implementation. The project includes sender-receiver interactions, frame generation, error simulation, and acknowledgment handling to mimic real-world transmission behavior.

Key Components and Implementation:

- Programming Language & Tools:
- The project was developed using Python with Tkinter for a GUI-based simulation.

Sender Module:

Divides the message into frames with sequence numbers.

Sends frames continuously within the sliding window limit.

Starts a timer for each frame sent.

Retransmits only frames that timeout or are negatively acknowledged.

Receiver Module:

Receives frames, checks sequence numbers.

Sends individual ACKs for correct frames.

Buffers out-of-order frames.

Reorders and delivers frames to the application layer when all previous frames are received.

Error Simulation: Random loss and corruption of frames and ACKs are introduced to simulate a real communication channel. This helps visualize how Selective Repeat efficiently handles such cases.

GUI Features :

Real-time display of sending/receiving frames.

Colored indicators for successfully received, lost, or retransmitted frames.

Timer and acknowledgment display.

Outcome:

The simulation successfully demonstrates how Selective Repeat enhances reliability by reducing unnecessary retransmissions. The system correctly identifies and retransmits only the lost or corrupted frames while maintaining proper order at the receiver's end. This project highlights the practical application of the SR protocol and provides a clear understanding of its benefits in real-time communication systems.

